

1. Introduction

Project Title

FreelanceHub – Connect Freelancers with Clients Seamlessly

Project Type

Individual Project

Role

Full Stack Developer

Project Description

FreelanceHub is a full-stack web application developed using the MERN stack that provides a platform for connecting clients with freelancers. The application allows users to register and log in, create and manage their profiles, explore freelance opportunities, and interact with the platform based on their role as a client or freelancer.

The project focuses on providing a simple and user-friendly environment where clients can find suitable freelancers for their requirements and freelancers can showcase their skills and find relevant opportunities.

2. Project Overview

2.1 Purpose

The purpose of **FreelanceHub** is to provide an online platform that connects clients with freelancers in a simple and efficient manner. Clients can find freelancers according to their requirements and skills, while freelancers can create their profiles and showcase their expertise to potential clients.

The application aims to simplify the process of finding freelance opportunities and hiring suitable freelancers through a centralized web platform.

2.2 Objectives

The main objectives of FreelanceHub are:

- To provide a platform for connecting clients and freelancers.
- To allow users to create and manage their accounts securely.
- To provide separate functionality for clients and freelancers.
- To allow freelancers to showcase their skills and professional profiles.
- To help clients discover suitable freelancers for their requirements.
- To provide an easy-to-use and responsive user interface.
- To store and manage application data using MongoDB.
- To implement secure user authentication and authorization.

2.3 Key Features

The major features of FreelanceHub include:

- **User Registration and Login** – Users can create an account and securely log in to the application.
- **Role-Based Users** – The platform supports different user roles, such as clients and freelancers.
- **Freelancer Profiles** – Freelancers can create profiles to showcase their skills and information.
- **Client Functionality** – Clients can use the platform to find and interact with freelancers according to their requirements.
- **Project Management** – The application provides functionality for managing freelance projects.

- **Reviews and Ratings** – Users can provide reviews to share their experience with freelancers.
- **Authentication and Authorization** – JWT-based authentication is used to protect user-specific functionality.
- **Responsive Interface** – The frontend is designed to work across different screen sizes.
- **Database Management** – User and application data are stored and managed using MongoDB.
- **REST API** – The frontend communicates with the backend through RESTful API endpoints.

2.4 Technologies Used

Category	Technology
Frontend	React.js
Build Tool	Vite
Backend	Node.js
Web Framework	Express.js
Database	MongoDB / MongoDB Atlas
ODM	Mongoose
Authentication	JSON Web Token (JWT)
Password Security	bcrypt
API Testing	Postman
Version Control	Git & GitHub
Frontend Hosting	Vercel
Backend Hosting	Render

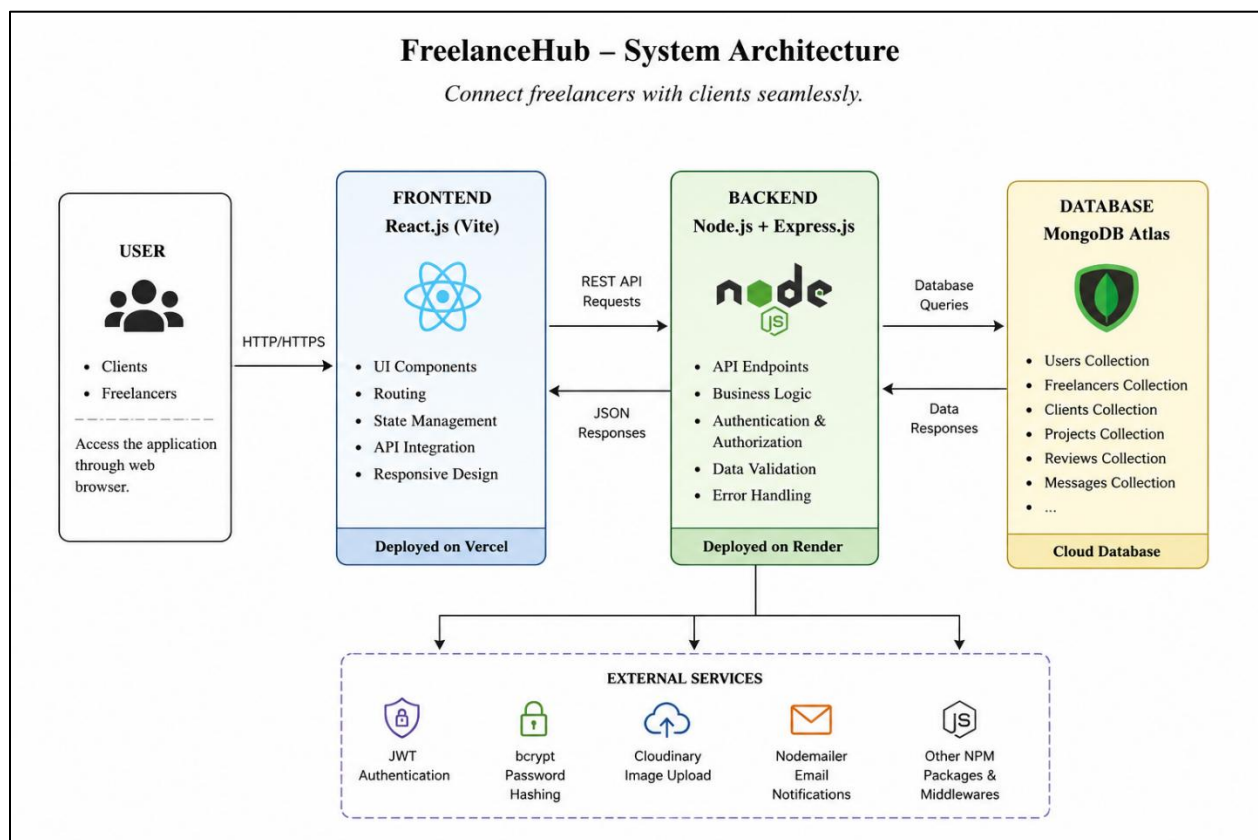
3. Architecture

FreelanceHub follows a **three-layer architecture** using the MERN stack. The application consists of a React frontend, a Node.js/Express.js backend, and a MongoDB database. The frontend communicates with the backend through RESTful APIs, while the backend handles business logic and database operations.

3.1 System Architecture

The overall architecture of FreelanceHub can be represented as:

User → React Frontend → REST API → Node.js + Express.js Backend → MongoDB Atlas



Architecture Flow

1. The user interacts with the FreelanceHub web application through the React frontend.
2. React sends HTTP requests to the backend REST APIs.
3. The Node.js and Express.js server receives and processes the requests.
4. Authentication and authorization are handled by the backend.
5. The backend uses Mongoose to communicate with MongoDB.
6. MongoDB stores application data such as users, freelancer information, client information, projects, and reviews.
7. The backend sends the requested data or response back to the React frontend.
8. The frontend displays the response to the user.

3.2 Frontend Architecture

The frontend of FreelanceHub is developed using **React.js** with **Vite** as the development and build tool.

The frontend is responsible for:

- Providing the user interface.
- Displaying user profiles and application data.
- Providing registration and login interfaces.
- Sending requests to backend REST APIs.
- Managing authentication tokens on the client side.
- Displaying appropriate pages and components based on user interaction.
- Providing a responsive interface for different screen sizes.

The React frontend communicates with the backend using HTTP requests. The API base URL is configured using environment variables so that the application can communicate with the deployed backend.

3.3 Backend Architecture

The backend is developed using **Node.js** and **Express.js**.

The backend is responsible for:

- Handling HTTP requests and responses.
- Providing RESTful API endpoints.
- Managing user registration and login.
- Performing authentication and authorization.

- Validating and processing user requests.
- Implementing application business logic.
- Performing CRUD operations on MongoDB.
- Managing users, freelancers, clients, projects, and reviews.
- Returning appropriate responses to the frontend.

The backend follows an organized structure using routes, controllers, models, and middleware to separate different responsibilities of the application.

3.4 Database Architecture

FreelanceHub uses **MongoDB** as its database and **MongoDB Atlas** for cloud database hosting.

Mongoose is used as the Object Data Modeling (ODM) library to define schemas and interact with MongoDB.

The database stores application-related information including:

- User accounts
- User roles
- Freelancer information
- Client information
- Project information
- Reviews and ratings
- Other application-related data

The backend communicates with MongoDB through Mongoose models. When a user performs an operation such as creating, updating, retrieving, or deleting data, the request is processed by the backend and the corresponding database operation is performed.

3.5 Deployment Architecture

The application is deployed using separate hosting services for the frontend and backend.

- **Frontend:** Deployed on Vercel.
- **Backend:** Deployed on Render.
- **Database:** Hosted on MongoDB Atlas.

The deployed frontend communicates with the deployed backend through the configured API URL.

4. Setup Instructions

4.1 Prerequisites

Before running FreelanceHub, the following software and tools are required:

- **Node.js** – Required to run the frontend and backend.
- **npm (Node Package Manager)** – Used to install and manage project dependencies.
- **MongoDB Atlas** – Used as the cloud-hosted database.
- **Git** – Used for version control and obtaining the project source code.
- **GitHub** – Used to host and manage the project source code.
- **Web Browser** – Required to access and use the application.
- **Postman** – Used for testing backend REST API endpoints.

4.2 Installation

Step 1: Clone the Repository

Clone the FreelanceHub project repository from GitHub:

```
git clone <repository-url>
```

Navigate to the project directory:

```
cd FreelanceHub
```

Step 2: Install Backend Dependencies

Navigate to the Server directory:

```
cd Server
```

Install the required dependencies:

```
npm install
```

Step 3: Configure Backend Environment Variables

Create a `.env` file inside the Server directory and configure the required environment variables.

Example:

```
PORT=5000
MONGO_URI=your_mongodb_atlas_connection_string
JWT_SECRET=your_jwt_secret
```

The MongoDB connection string should point to the MongoDB Atlas database used by the application.

Step 4: Install Frontend Dependencies

Open a new terminal and navigate to the Client directory:

```
cd Client
```

Install the frontend dependencies:

```
npm install
```

Step 5: Configure Frontend Environment Variable

Create a .env file inside the Client directory and configure the backend API URL.

Example:

```
VITE_API_URL=http://localhost:5000
```

For the deployed application, the value can be changed to the URL of the deployed backend server.

4.3 Environment Variables

The application uses environment variables to store configuration information separately from the source code.

Backend Environment Variables

Variable	Purpose
PORT	Specifies the port on which the backend server runs.
MONGO_URI	Stores the MongoDB Atlas connection string.
JWT_SECRET	Used for generating and verifying JWT authentication tokens.

Frontend Environment Variable

Variable	Purpose
VITE_API_URL	Specifies the base URL of the backend API.

4.4 Database Configuration

FreelanceHub uses MongoDB Atlas as its cloud database. The MongoDB Atlas connection string is stored in the backend environment variables and is used by the Node.js application to establish a connection with the database.

The database connection is managed through Mongoose.

4.5 Source Code Management

Git and GitHub are used for source-code management. The project source code is maintained in a GitHub repository, allowing version control and collaboration during development.

5. Folder Structure

FreelanceHub is organized into two main directories: **Client** and **Server**. The Client directory contains the React frontend, while the Server directory contains the Node.js and Express.js backend. The backend is further organized into controllers, middleware, models, routes, and configuration files.

5.1 Client Folder

The **Client** folder contains the frontend application developed using **React.js** and **Vite**.

- **src/** – Contains the main source code of the React application.
- **public/** – Contains public and static assets used by the application.
- **dist/** – Contains the production build generated by Vite.
- **.env** – Contains frontend environment variables.
- **index.html** – Main HTML entry point of the application.
- **package.json** – Contains frontend dependencies and scripts.
- **vite.config.js** – Contains Vite configuration.
- **eslint.config.js** – Contains ESLint configuration.

5.2 Server Folder

The **Server** folder contains the backend application developed using **Node.js** and **Express.js**.

5.2.1 Config

The **config** folder contains configuration-related files required by the backend application.

5.2.2 Controllers

The **controllers** folder contains the business logic for different modules of FreelanceHub.

- **adminController.js** – Handles administrator-related operations.
- **applicationController.js** – Handles project application operations.
- **authController.js** – Handles user registration and authentication.
- **chatController.js** – Handles chat-related operations.
- **dashboardController.js** – Handles dashboard-related operations.
- **freelancerController.js** – Handles freelancer-related operations.
- **notificationController.js** – Handles notification-related operations.
- **projectController.js** – Handles project-related operations.

5.2.3 Middleware

The **middleware** folder contains middleware used for authentication and role-based authorization.

- **authMiddleware.js** – Verifies authenticated users and their access tokens.
- **roleMiddleware.js** – Controls access based on the user's role.

5.2.4 Models

The **models** folder contains the Mongoose models used to interact with MongoDB.

- **User.js** – Defines the user data model.
- **Freelancer.js** – Defines the freelancer data model.
- **Project.js** – Defines the project data model.
- **Application.js** – Defines the project application data model.
- **Review.js** – Defines the review data model.
- **Chat.js** – Defines the chat data model.
- **Notification.js** – Defines the notification data model.

5.2.5 Routes

The **routes** folder contains the REST API routes of the backend.

- **adminRoutes.js** – Defines routes for administrator operations.
- **applicationRoutes.js** – Defines routes for project applications.
- **authRoutes.js** – Defines authentication-related routes.
- **chatRoutes.js** – Defines chat-related routes.
- **dashboardRoutes.js** – Defines dashboard-related routes.
- **freelancerRoutes.js** – Defines freelancer-related routes.
- **notificationRoutes.js** – Defines notification-related routes.
- **projectRoutes.js** – Defines project management routes.
- **reviewRoutes.js** – Defines review and rating routes.

5.2.6 Main Server Files

- **server.js** – Main entry point of the backend application.
- **test_backend.js** – Used for testing backend functionality.
- **.env** – Stores backend environment variables.
- **.env.example** – Provides an example structure for required environment variables.
- **package.json** – Contains backend dependencies and scripts.

6. Running the Application

FreelanceHub can be run locally by starting the frontend and backend applications separately. The frontend is developed using **React.js with Vite**, while the backend is developed using **Node.js and Express.js**.

6.1 Running the Backend

1. Open a terminal and navigate to the Server directory.

```
cd Server
```

2. Install the required backend dependencies.

```
npm install
```

3. Start the backend server.

```
npm start
```

The backend server starts on the configured port and provides the REST API services required by the frontend application.

6.2 Running the Frontend

1. Open a new terminal and navigate to the Client directory.

```
cd Client
```

2. Install the required frontend dependencies.

```
npm install
```

3. Start the Vite development server.

```
npm run dev
```

After starting the development server, Vite provides a local URL that can be opened in a web browser to access the FreelanceHub application.

6.3 Application Communication

The frontend communicates with the backend through REST API requests. The backend processes these requests and communicates with MongoDB Atlas to store and retrieve application data.

Application Flow:

React + Vite Frontend → Node.js + Express.js Backend → MongoDB Atlas

6.4 Deployed Application

FreelanceHub is deployed online using separate services for the frontend, backend, and database.

- **Frontend:** Vercel
- **Backend:** Render
- **Database:** MongoDB Atlas

The deployed React frontend communicates with the deployed backend through the configured API URL.

7. API Documentation

FreelanceHub provides RESTful APIs for authentication, freelancer management, project management, applications, reviews, chat, notifications, dashboards, and administrative operations.

The APIs use HTTP methods such as **GET, POST, PUT, PATCH, and DELETE**. Protected endpoints require user authentication, and selected endpoints additionally use role-based authorization.

7.1 Authentication APIs

Method	Endpoint	Description	Access
POST	/api/auth/register	Registers a new user.	Public
POST	/api/auth/login	Authenticates a user and logs them into the system.	Public
GET	/api/auth/me	Retrieves the currently authenticated user's information.	Authenticated

7.2 Admin APIs

All admin routes require authentication and **Admin** role authorization.

Method	Endpoint	Description	Access
GET	/api/admin/stats	Retrieves administrative statistics.	Admin
GET	/api/admin/users	Retrieves users for administrative management.	Admin
PATCH	/api/admin/users/:userId/status	Updates the status of a specific user.	Admin
GET	/api/admin/projects	Retrieves projects for administrative management.	Admin
DELETE	/api/admin/projects/:projectId	Deletes a project through the admin panel.	Admin
GET	/api/admin/applications	Retrieves project applications for administration.	Admin
GET	/api/admin/reviews	Retrieves reviews for administration.	Admin
DELETE	/api/admin/reviews/:reviewId	Deletes a review through the admin panel.	Admin

7.3 Freelancer APIs

Method	Endpoint	Description	Access
GET	/api/freelancers/	Retrieves all freelancers.	Authenticated
POST	/api/freelancers/	Creates a freelancer profile.	Freelancer
GET	/api/freelancers/:userId	Retrieves a freelancer profile using the user ID.	Authenticated
PUT	/api/freelancers/	Updates the authenticated freelancer's profile.	Freelancer

7.4 Project APIs

Method	Endpoint	Description	Access
GET	/api/projects/	Retrieves all open projects.	Authenticated
GET	/api/projects/:id	Retrieves a specific project.	Authenticated
POST	/api/projects/	Creates a new project.	Client
PUT	/api/projects/:id	Updates a project.	Client
DELETE	/api/projects/:id	Deletes a project.	Client
PATCH	/api/projects/:projectId/submit	Submits a completed project.	Freelancer
PATCH	/api/projects/:projectId/complete	Marks a submitted project as completed.	Client

7.5 Application APIs

Method	Endpoint	Description	Access
GET	/api/applications/my-applications	Retrieves applications submitted by the authenticated freelancer.	Freelancer
GET	/api/applications/client-received	Retrieves applications received by the authenticated client.	Client
POST	/api/applications/	Submits an application for a project.	Freelancer
GET	/api/applications/project/:projectId	Retrieves applications for a specific project.	Client / Admin
GET	/api/applications/:applicationId	Retrieves details of a specific application.	Authenticated
PATCH	/api/applications/:applicationId/status	Approves or rejects an application.	Client

7.6 Review APIs

Method	Endpoint	Description	Access
POST	/api/reviews/	Submits a review for a completed project.	Client
GET	/api/reviews/freelancer/:freelancerId	Retrieves reviews for a freelancer.	Authenticated
GET	/api/reviews/project/:projectId	Retrieves the review associated with a project.	Authenticated

7.7 Chat APIs

Method	Endpoint	Description	Access
POST	/api/chat/	Sends a message.	Authenticated
GET	/api/chat/:projectId	Retrieves all messages for a project.	Authenticated

7.8 Notification APIs

Method	Endpoint	Description	Access
GET	/api/notifications/	Retrieves notifications for the authenticated user.	Authenticated
GET	/api/notifications/unread-count	Retrieves the number of unread notifications.	Authenticated
PATCH	/api/notifications/:id/read	Marks a specific notification as read.	Authenticated
PATCH	/api/notifications/read-all	Marks all notifications as read.	Authenticated

7.9 Dashboard APIs

Method	Endpoint	Description	Access
GET	/api/dashboard/freelancer	Retrieves dashboard data for a freelancer.	Freelancer
GET	/api/dashboard/client	Retrieves dashboard data for a client.	Client

7.10 HTTP Methods Used

FreelanceHub uses standard HTTP methods to perform different operations:

- **GET** – Used to retrieve data.
- **POST** – Used to create new data.
- **PUT** – Used to update existing data.
- **PATCH** – Used for partial updates or status changes.
- **DELETE** – Used to remove data.

7.11 API Authentication

Protected APIs use authentication middleware to verify the user's authentication token. Some APIs additionally use role-based authorization to restrict access to specific user roles such as **Client, Freelancer, or Admin**.

8. Authentication

FreelanceHub implements authentication and authorization to protect user accounts and restrict access to role-specific functionality. Authentication is handled using **JSON Web Tokens (JWT)**, while passwords are securely processed using **bcrypt**.

8.1 User Registration

New users can register on the FreelanceHub platform by providing the required account information.

During registration:

1. The user submits the registration form.
2. The backend receives the registration request through the authentication API.
3. The user information is validated.
4. The password is securely hashed using bcrypt before being stored.
5. The user account is created in the MongoDB database.
6. The appropriate user role is stored with the account.

8.2 User Login

Registered users can log in using their account credentials.

The login process works as follows:

1. The user enters their login credentials.
2. The frontend sends the credentials to the backend login API.
3. The backend verifies the user information.
4. The password is compared with the stored hashed password using bcrypt.
5. After successful authentication, a JWT token is generated.
6. The token is returned to the frontend.
7. The frontend stores the authentication token and uses it when accessing protected APIs.

8.3 JSON Web Token (JWT)

JWT is used to maintain authenticated user access to protected resources.

The token is used by the backend to:

- Verify the identity of the user.
- Protect private API endpoints.
- Identify the authenticated user.
- Control access to user-specific functionality.

The authentication token is sent with requests to protected backend endpoints.

8.4 Authentication Middleware

FreelanceHub uses an authentication middleware to verify JWT tokens before allowing access to protected routes.

The middleware:

1. Receives the request.
2. Checks for the authentication token.
3. Verifies the token using the configured JWT secret.
4. Identifies the authenticated user.
5. Allows the request to continue if authentication is successful.
6. Rejects unauthorized requests when the token is missing or invalid.

8.5 Role-Based Authorization

In addition to authentication, FreelanceHub implements role-based authorization.

The application supports the following major roles:

- **Client**
- **Freelancer**
- **Admin**

Role-based middleware is used to restrict specific operations to authorized users.

For example:

- Only **Clients** can create, update, and delete projects.
- Only **Freelancers** can create and update freelancer profiles.
- Only **Freelancers** can submit applications.
- Only **Clients** can approve or reject applications.
- Only **Clients** can submit reviews for completed projects.
- Only **Admins** can access administrative operations.

8.6 Password Security

User passwords are not stored directly in plain text. Passwords are processed using **bcrypt** before being stored in the database.

This provides an additional layer of security for user credentials.

8.7 Authentication Flow

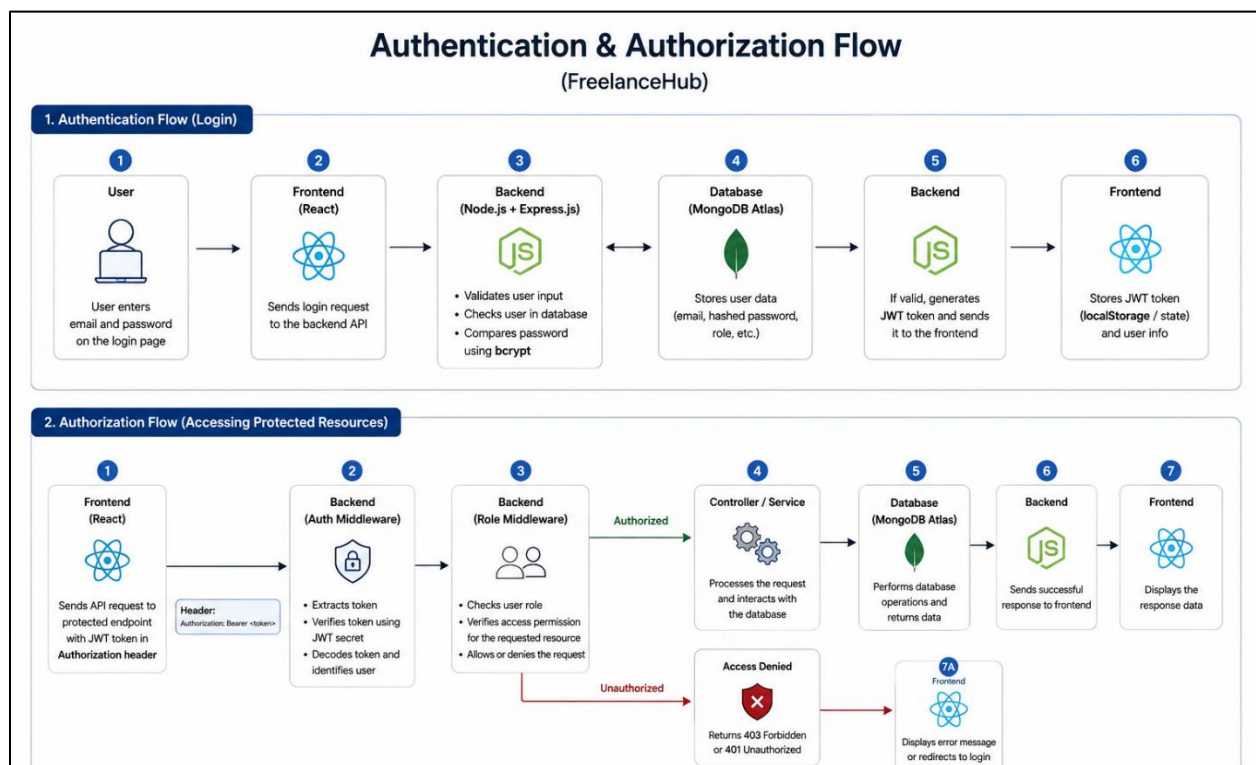
The authentication process can be summarized as:

User → Login → Backend → Verify Credentials → Generate JWT → Frontend → Protected API Requests → Authentication Middleware → Authorized Resource

8.8 Authorization Flow

For role-protected operations, the request passes through both authentication and role authorization middleware.

Request → JWT Authentication → Role Verification → Controller → Database Operation → Response



9. User Interface

FreelanceHub provides a user-friendly web interface designed to support different users and their respective activities on the platform. The interface is developed using React.js and provides separate functionality for Clients, Freelancers, and Administrators.

9.1 Landing Page

The landing page provides an introduction to the FreelanceHub platform and allows users to access the main features of the application.

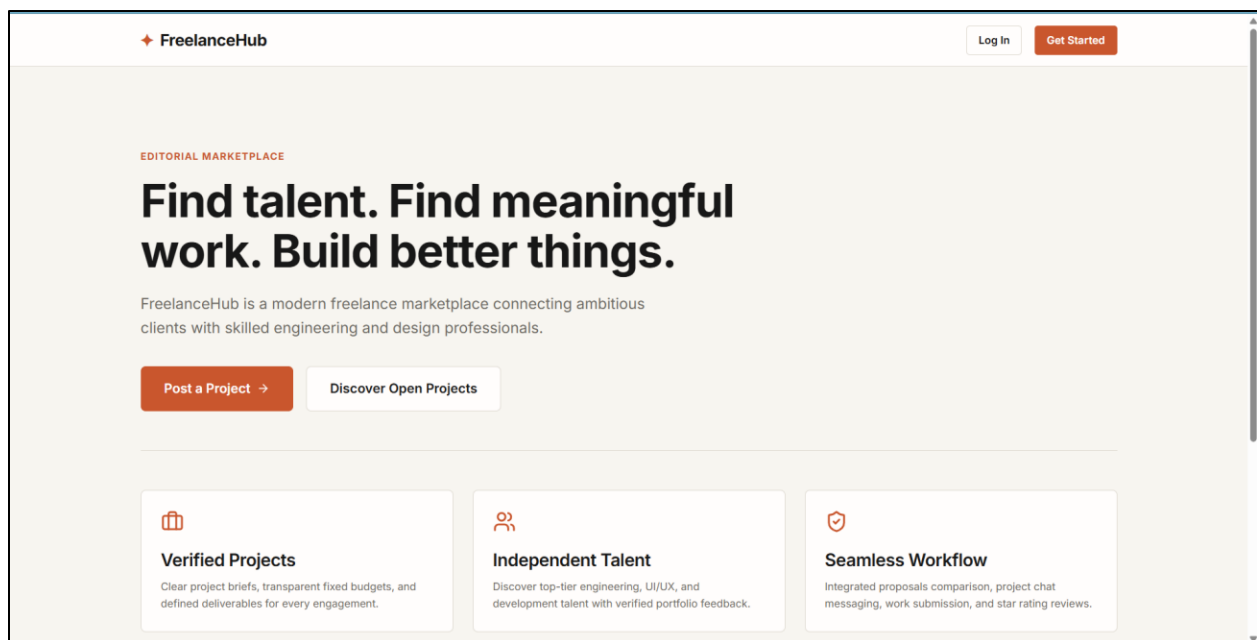
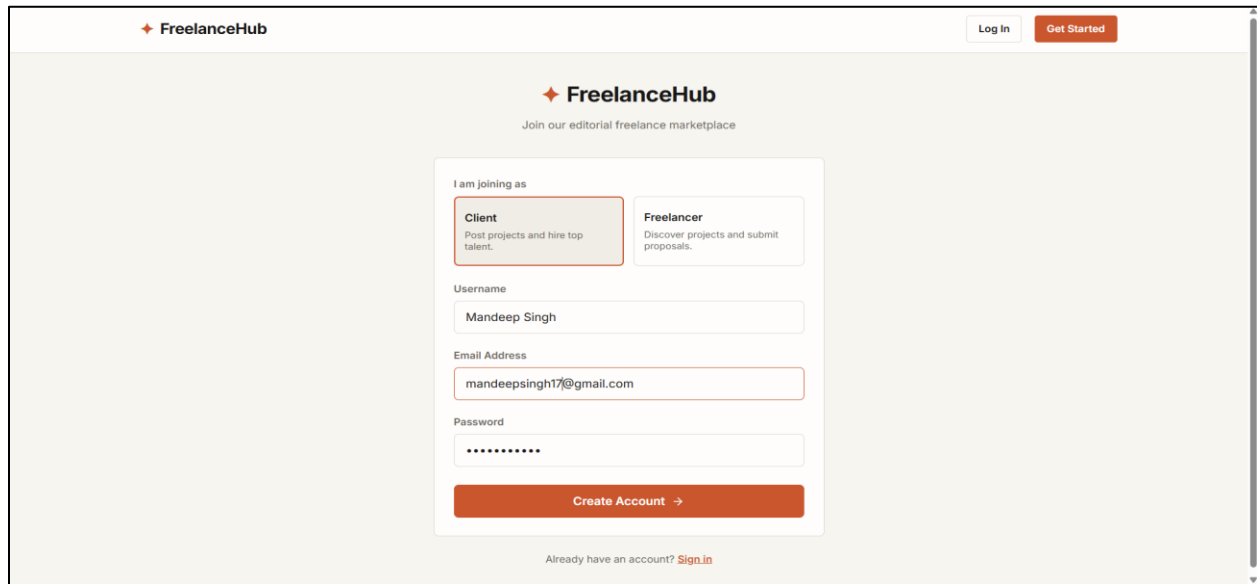


Figure 9.1: FreelanceHub Landing Page

9.2 User Registration

The registration page allows new users to create an account by providing the required information and selecting the appropriate user role.

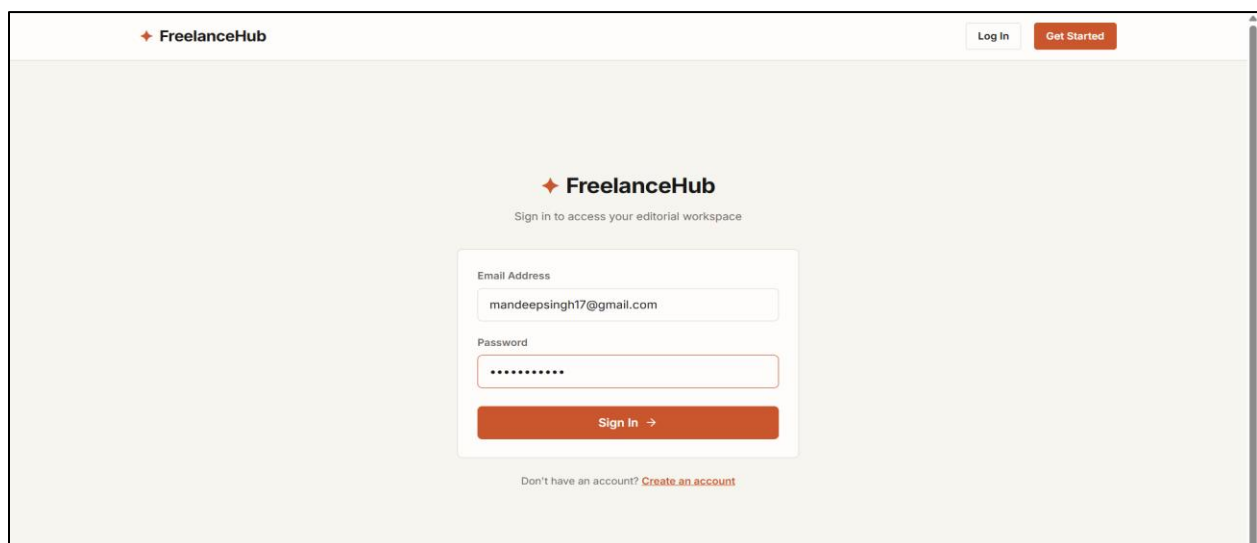


The screenshot shows the FreelanceHub registration page. At the top, there's a navigation bar with the FreelanceHub logo on the left and 'Log In' and 'Get Started' buttons on the right. The main content area features the FreelanceHub logo and the tagline 'Join our editorial freelance marketplace'. Below this, a registration form is centered. The form starts with a section titled 'I am joining as' containing two radio button options: 'Client' (with the description 'Post projects and hire top talent.') and 'Freelancer' (with the description 'Discover projects and submit proposals.'). The 'Client' option is selected. Below the radio buttons are three input fields: 'Username' (containing 'Mandeep Singh'), 'Email Address' (containing 'mandeepsingh17@gmail.com'), and 'Password' (containing masked characters). A red 'Create Account ->' button is at the bottom of the form. At the very bottom of the page, there is a link: 'Already have an account? [Sign In](#)'.

Figure 9.2: User Registration Page

9.3 User Login

The login page allows registered users to securely access their FreelanceHub account.



The screenshot shows the FreelanceHub login page. It has the same top navigation bar as the registration page. The main content area features the FreelanceHub logo and the tagline 'Sign in to access your editorial workspace'. Below this, a login form is centered. The form has two input fields: 'Email Address' (containing 'mandeepsingh17@gmail.com') and 'Password' (containing masked characters). A red 'Sign In ->' button is at the bottom of the form. At the very bottom of the page, there is a link: 'Don't have an account? [Create an account](#)'.

Figure 9.3: User Login Page

9.4 Client Dashboard

The Client Dashboard provides clients with features for managing projects, viewing applications, communicating with freelancers, and monitoring project activities.

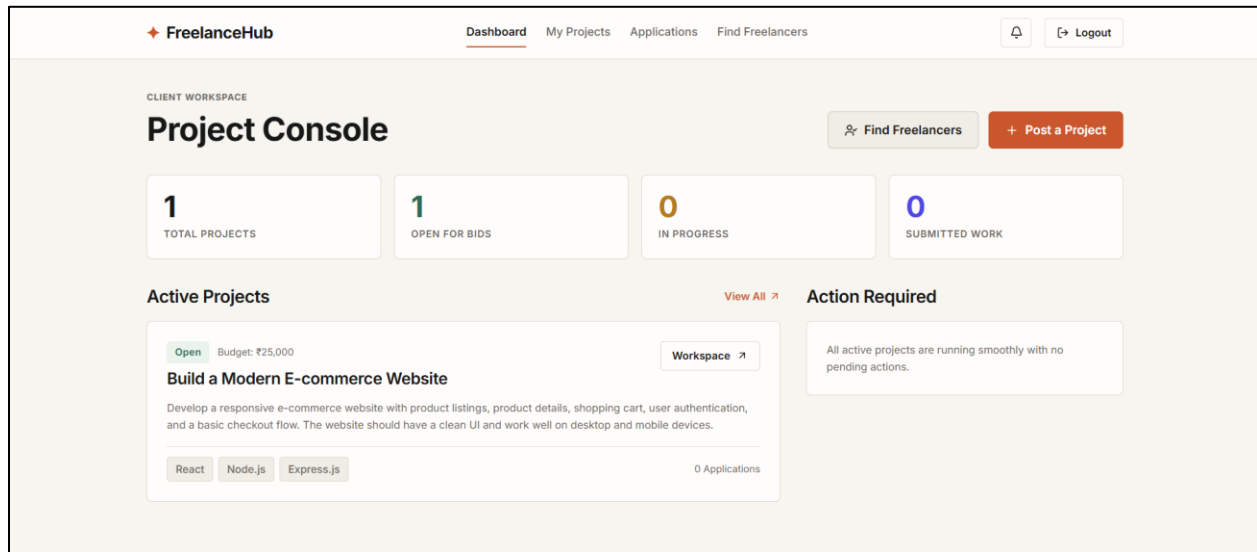


Figure 9.4: Client Dashboard

9.5 Freelancer Dashboard

The Freelancer Dashboard provides freelancers with features for discovering projects, managing applications, viewing projects, communicating with clients, and managing their profile.

Figure 9.5: Freelancer Dashboard

[Insert Freelancer Dashboard Screenshot Here]

9.6 Project Management

The project management interface allows clients to create and manage projects, while freelancers can view available projects and work on assigned projects.

Figure 9.6: Project Management Interface

[Insert Project Management Screenshot Here]

9.7 Freelancer Profile

The Freelancer Profile interface allows freelancers to create and manage their professional profile and provide relevant information about their skills and experience.

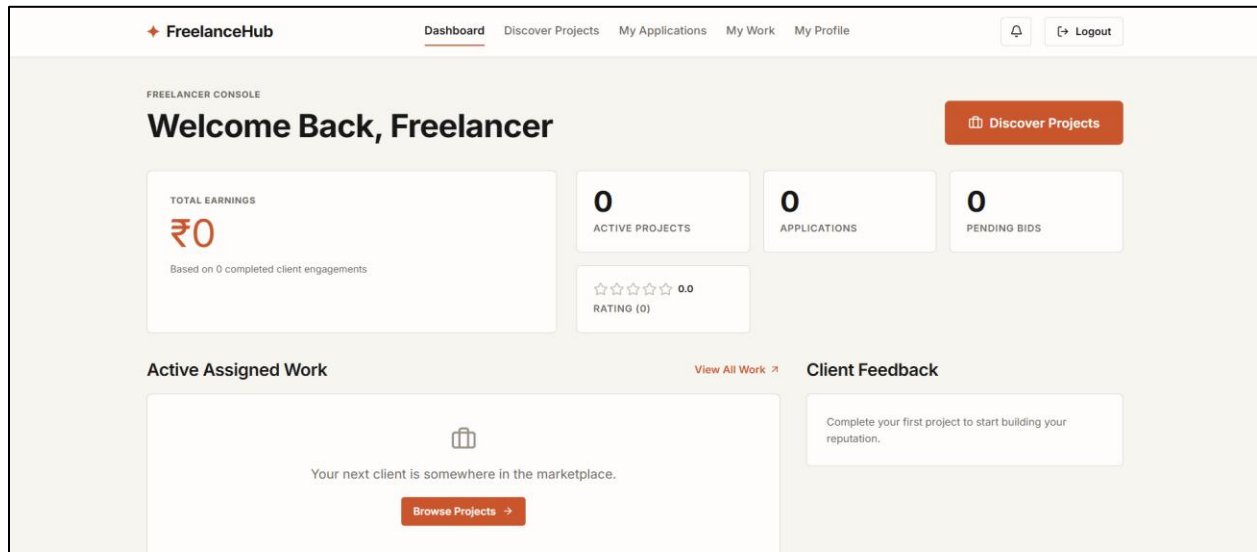
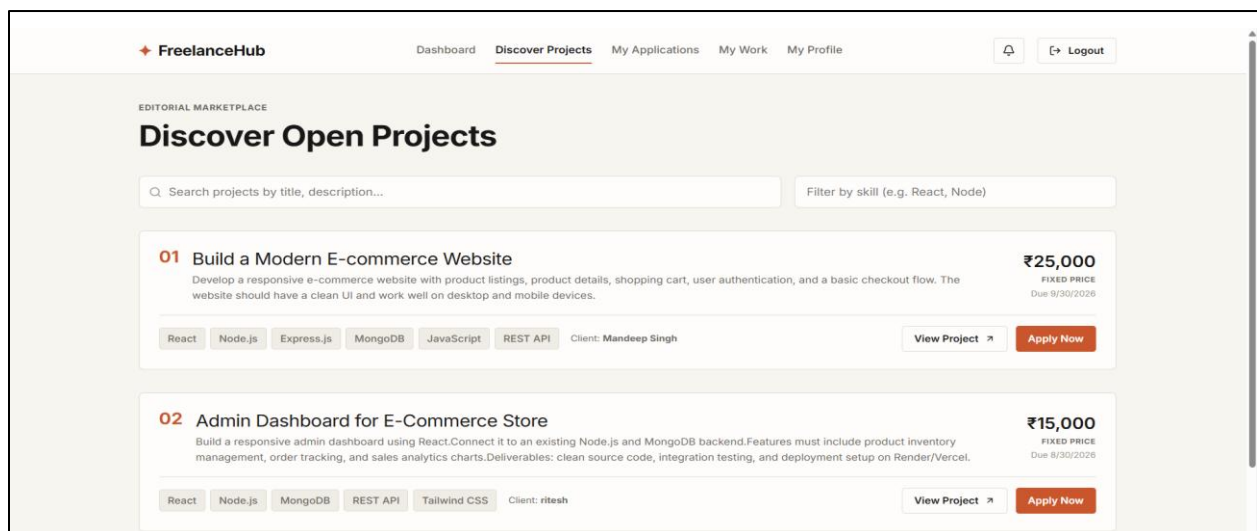


Figure 9.7: Freelancer Profile

9.8 Applications

The Applications interface allows freelancers to submit applications for projects and allows clients to view and manage received applications.



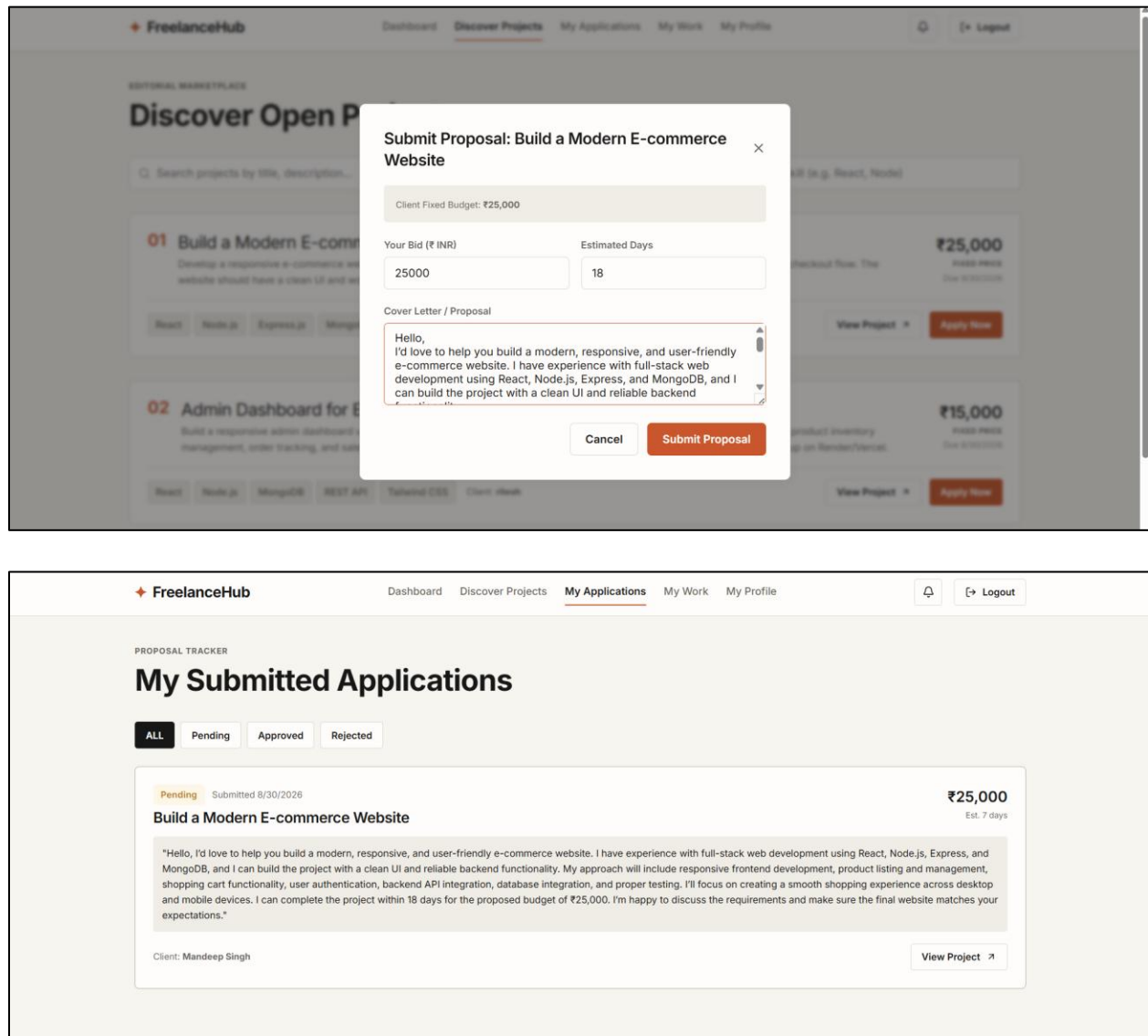


Figure 9.8.1: Freelancer Application Management Interface

The image displays two screenshots of the FreelanceHub client application management interface.

Top Screenshot: Project Overview and Proposals

The interface shows the project details for "Build a Modern E-commerce Website" (PROJECT / #B6A008). The budget is ₹25,000, the client is Mandeep Singh, and the deadline is 9/30/2026. The project overview describes the development of a responsive e-commerce website with product listings, product details, shopping cart, user authentication, and a basic checkout flow. The technologies listed are React, Node.js, Express.js, MongoDB, JavaScript, and REST API.

On the right, there is a section for "Proposals & Bids (1)". A proposal from Keshav Jadhav is shown, with a bid of ₹25,000 and an estimated time of 7 days. The proposal status is "Pending". The proposal text reads: "Hello, I'd love to help you build a modern, responsive, and user-friendly e-commerce website. I have experience with full-stack web development using React, Node.js, Express, and MongoDB, and I can build the project with a clean UI and reliable backend functionality. My approach will include responsive frontend development, product listing and management, shopping cart functionality, user authentication, backend API integration, database integration, and proper testing. I'll focus on creating a smooth shopping experience across desktop and mobile devices. I can complete the project within 18 days for the proposed budget of ₹25,000. I'm happy to discuss the requirements and make sure the final website matches your expectations."

Buttons for "Approve Candidate" and "Reject" are visible at the bottom of the proposal card.

Bottom Screenshot: Project Communication

This screenshot shows the "Project Communication" section. It indicates a direct line with Keshav Jadhav. The message area is currently empty, with a prompt: "No messages yet. Send a message to coordinate project deliverables."

At the bottom, there is a text input field labeled "Write a message..." and a red button with a paper plane icon for sending the message.

Figure 9.8.1: Client Application Management Interface

9.9 Chat and Notifications

FreelanceHub provides communication and notification features that allow users to communicate regarding projects and receive important application and project updates.

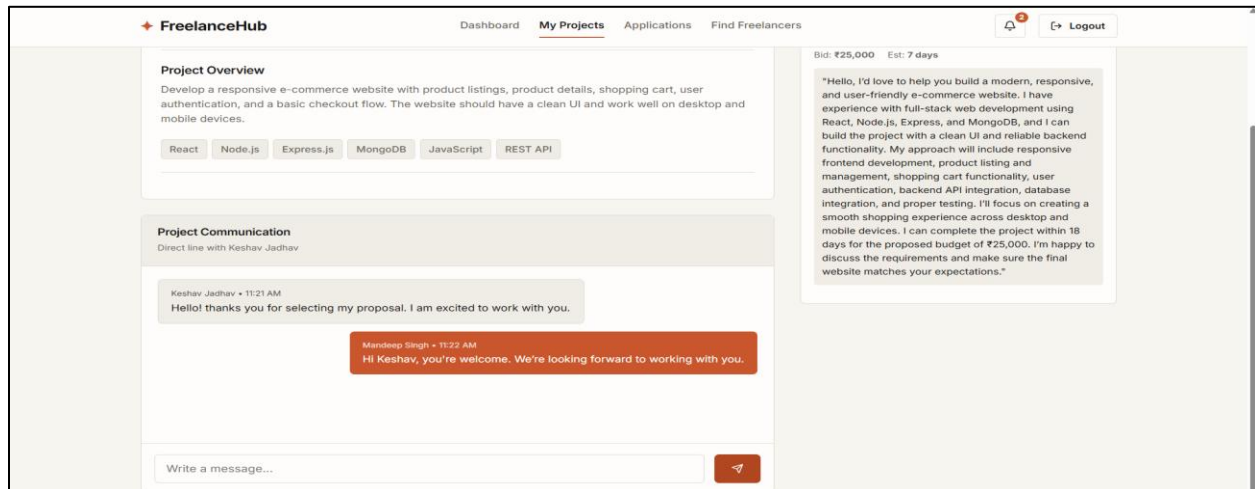


Figure 9.9: Chat and Notifications Interface

9.10 Reviews and Ratings

The Reviews and Ratings interface allows clients to provide feedback and ratings for completed projects and allows users to view freelancer reviews.

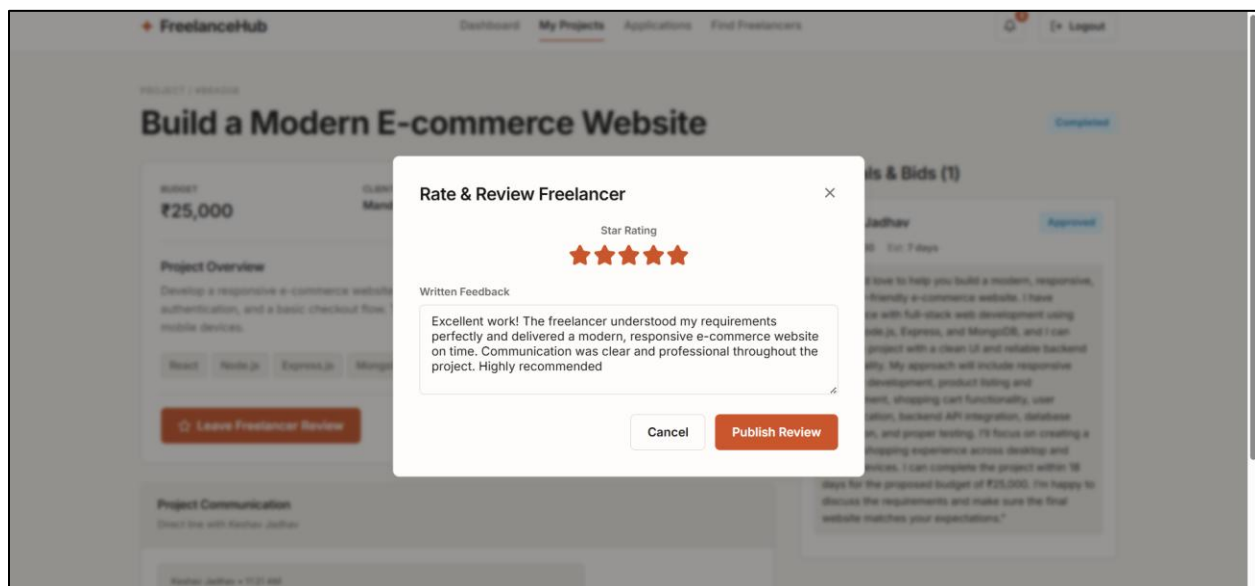


Figure 9.10: Reviews and Ratings Interface

10. Testing

Testing was performed to verify that the major functionalities of FreelanceHub work correctly and that the frontend, backend, authentication, database operations, and role-based features function as expected.

10.1 Testing Strategy

FreelanceHub was tested using a combination of functional testing and manual testing.

The major modules tested include:

- User registration and login
- Authentication and JWT-based access
- Client functionality
- Freelancer functionality
- Admin functionality
- Project creation and management
- Project applications
- Freelancer profiles
- Reviews and ratings
- Chat functionality
- Notifications
- Dashboard functionality
- MongoDB database operations
- Role-based authorization

10.2 Backend API Testing

The backend REST APIs were tested to verify that requests are correctly processed and appropriate responses are returned.

The API testing included:

- Testing GET, POST, PUT, PATCH, and DELETE requests.
- Verifying successful API responses.
- Testing authentication-protected endpoints.
- Testing role-based access restrictions.
- Testing invalid or unauthorized requests.
- Verifying database operations through API requests.

10.3 Authentication Testing

Authentication functionality was tested by performing the following checks:

- Registering a new user.
- Logging in with valid credentials.
- Testing login with invalid credentials.
- Accessing protected endpoints without authentication.
- Accessing protected endpoints with a valid JWT token.
- Verifying role-based access for Client, Freelancer, and Admin users.

10.4 Database Testing

MongoDB Atlas was used as the database for FreelanceHub.

Database-related testing included:

- Creating user records.
- Creating freelancer profiles.
- Creating and updating projects.
- Storing project applications.
- Storing reviews and ratings.
- Storing chat messages.
- Storing notifications.
- Retrieving and updating stored data.

10.5 Frontend Testing

The React frontend was manually tested to verify that:

- Pages load correctly.
- Forms accept and validate user input.
- Navigation works correctly.
- API responses are displayed correctly.
- User-specific dashboards display the appropriate information.
- Protected pages are accessible only to authorized users.
- The interface works correctly across different screen sizes.

10.6 Testing Tools

The following tools and technologies were used during development and testing:

- **Postman** – Used for testing backend REST APIs.
- **MongoDB Atlas** – Used to verify database records and database operations.
- **Browser Developer Tools** – Used to inspect frontend behavior, network requests, and errors.
- **VS Code** – Used for development, debugging, and code verification.
- **Git and GitHub** – Used for version control and maintaining project source code.

11. Demo

The deployed FreelanceHub application can be accessed through the following links:

Frontend: <https://getfreelance-hub.vercel.app>

Backend: <https://freelancehub-z6tr.onrender.com>

The frontend is deployed using Vercel, while the backend is deployed using Render. MongoDB Atlas is used as the cloud database.

12. Known Issues

During the development and deployment of FreelanceHub, some limitations and issues may be encountered. These issues are mainly related to the deployment environment, network connectivity, and application configuration.

12.1 Internet Dependency

FreelanceHub depends on an active internet connection for communication with the deployed backend and MongoDB Atlas database. Network interruptions may affect application functionality.

12.2 Backend Availability

The frontend application depends on the availability of the deployed backend server. If the backend service is unavailable, API-based features such as authentication, projects, applications, reviews, chat, and notifications may not function correctly.

12.3 Database Connectivity

The application depends on MongoDB Atlas for storing and retrieving application data. Database connectivity problems may prevent users from creating, updating, or retrieving data.

12.4 Environment Configuration

The application requires correctly configured environment variables for the frontend and backend. Incorrect API URLs, database connection strings, or authentication configuration may cause application errors.

12.5 Deployment Limitations

The deployed application may experience temporary delays or availability issues depending on the hosting environment and server resources.

12.6 Future Bug Fixes

Any additional bugs discovered during further testing or real-world usage can be documented and addressed in future versions of the application.

13. Future Enhancements

Although FreelanceHub provides the core functionality required to connect clients with freelancers, several improvements can be introduced in future versions to make the platform more powerful, scalable, and user-friendly.

13.1 Online Payment Integration

Integrate secure online payment services to allow clients to make payments to freelancers directly through the platform.

13.2 Advanced Search and Filtering

Implement advanced search and filtering options based on skills, project category, budget, experience, ratings, and availability to help clients find suitable freelancers more efficiently.

13.3 Real-Time Communication

Improve the existing chat functionality by introducing real-time messaging, typing indicators, message status, and instant notifications.

13.4 Enhanced Notification System

Expand the notification system to provide real-time updates for project applications, application status changes, messages, project submissions, reviews, and other important activities.

13.5 Freelancer Rating and Recommendation System

Develop an improved recommendation system that can suggest suitable freelancers to clients based on skills, previous projects, ratings, and experience.

13.6 Advanced Admin Features

Add additional administrative capabilities such as detailed analytics, user activity monitoring, project moderation, reporting, and platform performance statistics.

13.7 Mobile Application

Develop dedicated Android and iOS applications to allow clients and freelancers to access FreelanceHub through mobile devices.

13.8 Improved Security

Further enhance application security by implementing additional security measures such as improved input validation, rate limiting, secure API practices, and enhanced authorization mechanisms.

13.9 Scalability Improvements

Improve the backend architecture and deployment infrastructure to support a larger number of users, projects, applications, and concurrent requests.

13.10 AI-Based Features

Future versions could include AI-powered features such as freelancer recommendations, project suggestions, automated profile assistance, and intelligent matching between clients and freelancers.